

Drone System — 3-Node Architecture

MASTER ESP32 • SLAVE FC ESP32 • ESP32-CAM

Master ESP32-WROOM

Mission Controller

- WiFi + MQTT (TLS port 8883)
- HTTP — boot call + telemetry push
- BLE — setup only, killed after WiFi connects
- SPI — SD card (missions + queues)
- SD unmounts during flight, remounts between queue missions
- UART TX → slave FC (waypoints / commands)
- UART RX ← slave FC (NEXT_REQ / home / metrics)
- UART TX → ESP32-CAM (trigger / mode)
- UART RX ← ESP32-CAM (ACK / NACK)
- Button — short clicks fetch slots, 3s hold resets WiFi
- ESP-NOW RX — telemetry struct from slave
- Owns all 5 buffers — single source of truth

Slave FC ESP32-WROOM

Flight Controller

- UART RX ← master (waypoints / commands)
- UART TX → master (NEXT_REQ / home / metrics)
- MPU-6050 — gyro + accelerometer (I2C)
- GPS Nano 6M — position fix (UART2)
- BMP280 — barometer altitude hold (I2C)
- 4x ESC — 2822 1200KV motors (MCPWM PWM)
- PID loop — Core 1 dedicated (1kHz)
- Navigation loop — Core 0 (10Hz)
- ESP-NOW TX — telemetry struct to master
- First GPS fix saved as home → sent to master once
- Clones: Buffer 1 (target) + Buffer 3 (mission state)
- Dumb node — no mission logic, only executes points

ESP32-CAM (OV2640)

Camera Node

- UART RX ← master (trigger commands)
- UART TX → master (ACK / NACK)
- OV2640 — 8-bit parallel interface
- Onboard SD card — photo storage
- Buffer mode 1 / 2 / 3 from master
- Matrix mode — shoot on each waypoint arrival
- Polygon mode — shoot on interval timer
- Clones: Buffer 3 (mission state) + Buffer 5 (camera state)
- Passive node — only reacts to master commands
- ACK (0x06) on success, NACK (0x15) if busy

Master ↔ Slave — UART

TX: waypoints, RTH, LAND, ABORT commands

RX: NEXT_REQUEST, home point, telemetry metrics

Master ↔ Camera — UART

TX: SHOOT, SET_MODE, START_INTERVAL, STOP_INTERVAL

RX: ACK (0x06) or NACK (0x15)

Master ↔ Cloud — WiFi

MQTT TLS 8883 — missions, commands, state, telemetry

HTTP — boot auth, telemetry push, slot fetch

Master ↔ App — BLE

Setup only — WiFi credential provisioning

Killed after WiFi connects to free 60KB RAM

Slave → Master — ESP-NOW

Telemetry struct: lat, lng, alt, speed, heading, battery, signal

One-way, fires every 3 seconds from slave

Buffer Architecture

MASTER OWNS ALL 5 BUFFERS • SLAVE AND CAMERA RECEIVE CLONES • ONLY MASTER WRITES

Buffer 1 Current Target Point

Cloned to: Slave FC

lat float32 lng float32 alt uint8 meters

The single waypoint the slave is currently flying toward. Updated by master every time a new point is served. Slave reads this clone to know its active target.

Buffer 2 Home Point

Master only — never cloned

lat float32 lng float32 alt uint8 meters

Set exactly once when slave sends its first valid GPS fix via UART on boot. Never overwritten. On RTH, master serves this as the next waypoint — slave treats it like any normal point.

Buffer 3 Mission State

Cloned to: Slave FC + Camera

mission_type uint8 (1=matrix, 2=polygon) current_index uint16 total_points uint16

status IDLE / FLYING / RTH / LANDING / ABORTED

Drives everything — point serving rhythm for slave and camera trigger mode for ESP32-CAM. Both nodes clone this to know what phase the mission is in.

Buffer 4 Slave Telemetry

Master only — pushed to MQTT + HTTP

lat float32 lng float32 alt float32 speed float32 heading float32 battery uint8 % signal_strength int

Filled by ESP-NOW receive callback from slave every 3 seconds. Master pushes this to both MQTT telemetry topic and HTTP telemetry endpoint.

Buffer 5 **Camera State**

Cloned to: ESP32-CAM

buffer_mode uint8 (1/2/3) trigger_flag bool interval_ms uint32 last_shot_index uint16

Set at mission start based on mission_type. Sent to camera as SET_MODE command. Drives whether camera shoots on waypoint arrival (matrix) or on interval timer (polygon).

Clone Distribution Matrix

Buffer	Master ESP32	Slave FC	ESP32-CAM
Buffer 1 — current target point	WRITE	READ clone	—
Buffer 2 — home point	WRITE	—	—
Buffer 3 — mission state	WRITE	READ clone	READ clone
Buffer 4 — slave telemetry	WRITE	—	—
Buffer 5 — camera state	WRITE	—	READ clone

Mission Execution Flow

BOOT • HOME DETECTION • POINT SERVING • CAMERA TRIGGER • RTH • EMERGENCY LAND • QUEUE

01

Boot

Master boots → HTTP POST to boot URL with MAC address → receives userID + droneID → builds MQTT base topic → connects MQTT over TLS

02

SD mount

SPI bus init → SD.begin() → ensures /missions and /queues directories exist → sdReady flag set

03

Slave first GPS fix

Slave FC boots → PID loop starts on Core 1 → GPS lock acquired → first valid fix saved internally as home → UART packet sent to master (0x07 HOME_POINT + lat + lng + alt)

04

Home stored in master

Master receives 0x07 HOME_POINT via UART → stores lat/lng/alt in Buffer 2 → never overwritten for the entire session

05

Mission arrives via MQTT

App publishes JSON payload to drone/{id}/sd/save_mission → master saves full JSON to SD at /missions/{id}.json → publishes ACK to /sd/ack

06

Execute command received

App publishes to /sd/execute_mission with mission id → master opens file from SD → reads mission_type and full flight_path array into RAM struct array

07

SD unmounts — flight RAM only

All waypoints in RAM → SD.end() called → SPI bus fully released → no SD activity during flight → RAM holds entire mission for zero-latency point serving

08

Camera mode configured

mission_type read → Buffer 5 buffer_mode set (1 or 2) → master sends 0xA2 SET_MODE to ESP32-CAM via UART → camera acknowledges with 0x06 ACK

09

First point served

Master sends 12-byte UART packet: 0xAA + 0x01 + lat + lng + alt + checksum → slave receives → updates navigation target → Buffer 1 clone updated → drone begins flying to point[0]

10

Point-by-point loop

Slave arrives within threshold → sends 0x06 NEXT_REQUEST → master increments current_index → serves point[n+1] → repeat until last point reached

11

Camera trigger logic

Matrix (type 1): on each 0x06 NEXT_REQUEST received, master sends 0xA1 SHOOT to camera before serving next point. Polygon (type 2): master starts interval FreeRTOS timer when leg begins, fires 0xA1 SHOOT every N seconds, stops timer on next NEXT_REQUEST

12

Queue — next mission

Last point of mission reached → if queue active: SD remounts → next mission JSON loaded into RAM → SD unmounts again → mission_type re-evaluated → camera mode updated → point[0] of new mission served

13

RTH triggered

App sends RTH via MQTT or BLE → master sets rth_requested flag → on next NEXT_REQUEST from slave (or immediately): serves Buffer 2 home point as a normal 0x01 waypoint UART packet → slave navigates home like any other point → sends 0x08 ARRIVED_HOME when done

Emergency land

14

App sends LAND → master reads Buffer 1 current target lat/lng → sends same coordinates but with alt=0 as 0x01 waypoint → slave descends vertically to ground at current XY position → sends 0x09 LANDED when complete

UART Protocol + Command Reference

MASTER ↔ SLAVE • MASTER ↔ CAMERA • MQTT COMMANDS • BUTTON BEHAVIOR

Master → Slave UART Packet (12 bytes fixed)

Byte(s)	Field	Type	Description
0	START	0xAA	Frame start marker — always 0xAA
1	CMD	uint8	0x01 waypoint 0x02 RTH 0x03 LAND 0x04 ABORT
2–5	lat	float32	Target latitude (IEEE 754, little-endian)
6–9	lng	float32	Target longitude (IEEE 754, little-endian)
10	alt	uint8	Target altitude in meters — 0 means land in place
11	CHECKSUM	uint8	XOR of bytes 1 through 10

Slave → Master UART Responses

Byte	Name	Payload	When sent
0x06	NEXT_REQUEST	none	Waypoint arrival confirmed — master must send next point
0x07	HOME_POINT	lat(4) + lng(4) + alt(1)	First valid GPS fix on boot — sent once only
0x08	ARRIVED_HOME	none	RTH complete — drone is at home position
0x09	LANDED	none	Emergency land complete — altitude reached 0

Byte	Name	Payload	When sent
0x15	NACK	none	Could not execute last command received

Master → Camera UART Commands

Byte	Command	Extra bytes	Description
0xA1	SHOOT	none	Take one photo now — save to SD — reply ACK or NACK
0xA2	SET_MODE	1 byte mode	Set buffer mode 1/2/3 based on mission_type — sent at mission start
0xA3	START_INTERVAL	1 byte seconds	Begin interval shooting — polygon mode — fires every N seconds
0xA4	STOP_INTERVAL	none	Stop interval shooting — called when polygon leg ends

MQTT Command Topic — drone/{droneID}/command

Command value	Master action
ARM	Publish state ARMED to status topic
DISARM	Publish state IDLE
TAKEOFF	Publish state FLYING — slave handles altitude from mission point alt value
LAND	Read Buffer 1 lat/lng → send with alt=0 to slave via UART
RTH	Set rth_requested flag → serve Buffer 2 home point as next UART waypoint
ABORT	Publish state LANDING → send LAND command → stop mission serving → clear mission RAM
LOITER	Publish state FLYING → pause point serving — drone hovers at current position

Boot Button Behavior (GPIO 0)

Action	Behavior
1 click within 2s window	Fetch mission slot 1 via HTTP GET to slot URL
2 clicks within 2s window	Fetch mission slot 2
3 clicks within 2s window	Fetch mission slot 3
4 clicks within 2s window	Fetch mission slot 4
5 clicks within 2s window	Fetch mission slot 5
Hold 3 seconds	Clear WiFi credentials from NVS → restart → enter BLE setup mode